

Canonical Written Interview

1. Engineering Experience

Provide a high-level overview of the types of projects that you have worked on including the technical stack used in those projects.

Most of my work has been on backend-heavy full-stack systems for AI communication, real-time conferencing, healthcare workflows, analytics platforms, and web/mobile applications.

My main stack includes Python, Django, Django REST Framework, FastAPI, PostgreSQL, MySQL, Redis, REST APIs, GraphQL, gRPC, WebSockets, Docker, Kubernetes, and AWS. I have also worked with React, Next.js, React Native, TypeScript, Redux, D3.js, and accessibility-focused UI development.

Describe the most complex/largest project(s) that you have worked on that involves Python/Go, SQL, and NoSQL.

The most complex project I worked on was an AI-powered multilingual conferencing platform. It used Python, Django, Django REST Framework, FastAPI, WebRTC, LiveKit, REST APIs, gRPC, WebSockets, PostgreSQL, MySQL, Redis, OpenSearch, Docker, Kubernetes, and AWS.

The platform supported scheduled meetings, instant calls, secure join links, participant controls, real-time transcription, live translation, captions, AI summaries, recordings, and admin workflows. PostgreSQL and MySQL handled meetings, users, permissions, transcripts, and analytics. Redis supported caching and async processing, while OpenSearch supported transcript and meeting search.

Python is my strongest production language. I have not used Go as deeply in production, but I am comfortable learning and working with it where needed.

Describe your proudest achievement as a software engineer.

My proudest achievement was helping build a real-time AI meeting platform that reduced manual meeting documentation by about 70%.

The system combined live communication, transcription, translation, summaries, secure access, cloud deployment, and search. I am proud of it because it solved a real problem for users and made meetings easier to follow, document, and review.

Describe your experience working on open-source projects and the lessons learnt.

I have not been a major maintainer of a large public open-source project. Most of my open-source experience comes from building production systems with Linux, Python, Django, FastAPI, PostgreSQL, Redis, Docker, Kubernetes, OpenSearch, React, LangChain, and related tools.

I have worked with open-source documentation, issues, source code, debugging, and internal improvements around these tools. The main lesson I learned is that clear documentation, small reproducible examples, and focused contributions are very important in open-source work.

Describe your experience architecting and engineering a large system with many services and the lessons learnt.

I worked on a multilingual conferencing platform with services for authentication, meeting management, participant control, media/session coordination, transcription, translation, summarization, TTS, transcript storage, search, analytics, and admin workflows.

The main design decision was separating real-time meeting behavior from background AI processing. Redis, async processing, caching, and search indexing helped keep the system stable and responsive.

The main lesson was that service boundaries should follow real operational needs. Services that scale, fail, or deploy differently need clear separation.

Describe your experience/journey with Linux.

Linux has been part of my daily engineering work for backend development, server environments, Docker, Kubernetes, AWS deployments, shell scripting, log analysis, process inspection, networking checks, and production troubleshooting.

Most systems I worked on were deployed on Linux-based infrastructure. I am comfortable with command-line workflows, environment variables, permissions, SSH, service logs, containers, package installation, and basic system debugging.

Describe your DevOps/DevSecOps, CI/CD or other related automation experience.

I have worked with GitHub Actions, Jenkins, Docker, Kubernetes, AWS EKS, ECR, EC2, RDS, S3, CloudFront, Route 53, IAM, Secrets Manager, CloudWatch, and Cloudflare.

My CI/CD work included running tests, building Docker images, managing deployments, checking environment configuration, and improving release reliability.

My DevSecOps experience includes JWT, OAuth 2.0, OIDC, RBAC, rate limiting, Cloudflare WAF, encrypted storage, audit logging, secure API design, secrets management, and access control.

Describe your SysAdmin, SRE or other such operational experience of a large Cloud infrastructure.

I have not held a pure SRE title, but I have supported production systems running on AWS and Kubernetes.

My operational work included service monitoring, deployment troubleshooting, log analysis, background job monitoring, database performance work, caching, WebRTC/media infrastructure support, and cloud service debugging.

Describe your experience in driving software quality improvements and the key lessons learnt.

I have improved software quality through automated tests, code reviews, pull requests, better API contracts, typed interfaces, clearer architecture, and disciplined release workflows.

I have used Pytest, Unittest, Jest, React Testing Library, Playwright, and TDD-style practices. The main lesson I learned is that quality is not only test coverage. Quality means the system is understandable, reliable, and safe to change.

Describe your experience in driving documentation improvements and the key lessons learnt.

I have written and improved documentation for APIs, local setup, deployment steps, environment variables, third-party integrations, troubleshooting, and service responsibilities.

The main lesson I learned is that documentation should be updated together with code changes. If an API, deployment process, or operational behavior changes, the documentation should change at the same time.

Describe your experience in driving UI/UX improvements and the key lessons learnt.

I have worked on UI/UX improvements using React, Next.js, React Native, TypeScript, Storybook, D3.js, Three.js, responsive design, and WCAG-focused accessibility.

My UI work included meeting flows, live captions, AI summaries, caregiver dashboards, and role-based interfaces. The main lesson I learned is that backend quality strongly affects UI quality. Clear APIs, fast responses, and reliable state changes make the user experience much better.

Describe your experience in driving software performance improvements and the key lessons learnt.

I have improved performance in databases, caching, search, async processing, frontend rendering, and cloud deployment.

In one analytics platform, I helped improve biomechanical data processing by 4x. In the meeting platform, I improved transcript retrieval, call history access, search, and analytics performance using indexing, partitioning, Redis caching, and OpenSearch.

The main lesson I learned is to measure first. Good performance work starts with profiling, logs, metrics, and a clear before-and-after comparison.

Describe your experience in driving security improvements and the key lessons learnt.

I have worked on security improvements around authentication, authorization, API access, healthcare data workflows, meeting access, and cloud deployment.

My experience includes JWT, OAuth 2.0, OIDC, RBAC, rate limiting, Cloudflare WAF, encrypted storage, audit logging, secure join links, secure API design, and secrets management.

The main lesson I learned is that security must cover the full data lifecycle: creation, transmission, storage, search, access, retention, and deletion.

2. Education

How did you rank in your high school, in your final year in maths and hard sciences? Which was your strongest?

I was strongest in technical and analytical subjects. Mathematics was my strongest area because I enjoyed structured problem solving and logical reasoning.

How did you rank in your high school, in your final year in languages and the arts? Which was your strongest?

I was solid in language-related subjects, but my strongest direction was technical. Written communication was the most useful area for me because it later helped with documentation, pull requests, and remote engineering communication.

Please state your high school graduation results or university entrance results, along with the system used, and how to understand those.

I completed high school under the Belarusian grading system. I do not have the exact official score available now. My strongest areas were mathematics, technical subjects, and analytical problem solving.

What sort of high school student were you? Outside of class, what were your interests and hobbies? What would your high school peers remember you for, if we asked them?

I was a quiet, practical, and curious student. I liked computers, technology, logic, and solving practical problems. My peers would probably remember me as reliable, calm, and helpful with technical or logical topics.

Which university and degree did you choose? What other universities did you consider, and why did you select that one?

I chose a B.S. in Computer Science at Belarusian Trade and Economics University of Consumer Cooperatives. I chose computer science because I wanted a practical path into software engineering, databases, systems, and web applications.

At university, did you do particularly well at any area of your degree?

I did best in programming, databases, and analytical problem solving. Those areas matched the type of work I later did professionally, including API design, database modeling, backend services, and performance improvement.

Overall, what was your degree result and how did that reflect on your ability?

I successfully completed a B.S. in Computer Science from Belarusian Trade and Economics University of Consumer Cooperatives under the Belarusian university grading system. The degree reflects a solid foundation in programming, databases, software design, and analytical problem solving.

In high school and university, what did you achieve that was exceptional?

My strongest achievement was building the technical foundation and self-learning discipline that allowed me to move into professional software engineering and keep growing across backend, cloud, infrastructure, and AI-related systems.

What leadership roles did you take on during your education?

My leadership during education was mostly informal. I helped classmates with technical topics, contributed to group assignments, and was known as organized and dependable.

3. Context

Outline your thoughts on the mission of Canonical. What is it about the company's purpose and goals which is most appealing to you? What do you see as risky or unappealing?

Canonical appeals to me because it works on Linux, open source, cloud infrastructure, Kubernetes, automation, security, and developer platforms. The most appealing part is the mix of open-source technology and enterprise-level responsibility. That fits my experience with backend systems, Kubernetes, AWS, CI/CD, observability, secure APIs, and production support. The main risk is the broad product scope and high expectations in a remote-first environment. That requires strong ownership, clear communication, and good documentation.

Who are Canonical's key competitors, and how should Canonical set about winning?

Canonical's competitors include Red Hat, SUSE, Oracle Linux, Debian-based options, cloud provider Linux distributions, Red Hat OpenShift, SUSE Rancher, VMware Tanzu, EKS, GKE, and AKS. Canonical can win by focusing on trusted open source, operational simplicity, strong security, clear documentation, and predictable enterprise support.

Why do you most want to work for Canonical?

I want to work for Canonical because the role matches the work I want to keep doing: backend engineering, Linux-based infrastructure, Python services, distributed systems, cloud automation, security, and production reliability. My recent experience with Python, Django, FastAPI, Kubernetes, AWS, PostgreSQL, Redis, OpenSearch, CI/CD, observability, and secure APIs fits well with this role.

What would you most want to change about Canonical?

From the outside, I would not assume I know what Canonical should change. The area I would care most about improving is developer and operator experience: clearer onboarding, better documentation, practical examples, reliable automation, and easier troubleshooting.

What gets you most excited about this role?

I am most excited about working on backend systems that matter at infrastructure scale. I enjoy service design, APIs, data modeling, performance, security, automation, deployment, and production reliability. This role fits that direction well.

Aliaksei Krents